

JAVA Data Types

Operations and Conversions

Numerical Data Types

Name	Range	Storage Size
Byte	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
short	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
int	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
long	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

Numeric Operators

Name	Meaning	Example	Result
+	Addition	<code>34 + 1</code>	35
-	Subtraction	<code>34.0 - 0.1</code>	33.9
*	Multiplication	<code>300 * 30</code>	9000
/	Division	<code>1.0 / 2.0</code>	0.5
%	Remainder	<code>20 % 3</code>	2
<code>Math.pow</code>	Exponent	<code>Math.pow(2, 3)</code>	8

Remainder Operator

The **%** operator is often used for positive integers, but it can also be used with negative

$$5 \% 2 \rightarrow 1$$

$$5 \% (-2) \rightarrow 1$$

$$(-5) \% 2 \rightarrow -1$$

$$(-5) \% (-2) \rightarrow -1$$

Remainder is very useful in programming. **For example:**

- An even number % 2 is always 0
- An odd number % 2 is always 1.
- Suppose today is Saturday and you and your friends are going to meet in 10 days. What day is in 10 days?

Day 6 in a week is Saturday

A week has 7 days

$$(6 + 10) \% 7 \text{ is } 2$$

Day 2 in a week is Tuesday

Note: Day 0 in a week is Sunday

After 10 days

Augmented Assignment Operators

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>

`x /= 4 + 5.5 * 1.5;` is same as `x = x / (4 + 5.5 * 1.5);`

Note: There are no spaces in the augmented assignment operators.

`+` `=` *should be* `+=`.

Increment and Decrement Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>	<i>Example (assume i = 1)</i>
++var	preincrement	Increment var by 1 , and use the new var value in the statement	int j = ++i; // j is 2, i is 2
var++	postincrement	Increment var by 1 , but use the original var value in the statement	int j = i++; // j is 1, i is 2
--var	predecrement	Decrement var by 1 , and use the new var value in the statement	int j = --i; // j is 0, i is 0
var--	postdecrement	Decrement var by 1 , and use the original var value in the statement	int j = i--; // j is 1, i is 0

Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read.

Mathematical Functions

Java provides many useful methods in the **Math** class for performing common mathematical functions.

Examples :

Math.PI

Math.pow(2, 3) returns 8.0

Math.pow(3, 2) returns 9.0

Math.sqrt(4) returns 2.0

Math.sin(Math.PI/2) returns 1.0

Math.max(2, 3) returns 3

Math.min(2.5, 3.6) returns 2.5

Math.abs(-2) returns 2

Math.round(2.6) returns 3

Math.random() Generates a random double value greater than or equal to 0.0 and less than 1.0.

Can be used as:

a+(int) (Math.random() * b) ;

To return a random integer between **a** and **a+b-1**

Type Conversion Rules

Consider: `byte i = 100;`
`long k = i * 3 + 4;`
`double d = i * 3.1 + k / 2;`

When performing a binary operation involving two operands of different types, Java automatically converts the operand as follow.

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.

Java automatically widen a type, but you must narrow a type explicitly.

Type Casting

Implicit casting: `double d = 3;` (type widening)

Explicit casting: `int i = (int)3.0;` (type narrowing)
`int i = (int)3.9;` (Fraction part is lost)

What is wrong? `int x = 5 / 2.0;`

What about? `int x = 5 / 2;`

`double x = 5 / 2;`

range increases



byte, short, int, long, float, double

Type Casting

Implicit casting: `double d = 3;` (type widening)

Explicit casting: `int i = (int)3.0;` (type narrowing)
`int i = (int)3.9;` (Fraction part is lost)

What about? `int sum = 0;`
`sum += 4.5;`

is equivalent to: `sum = (int)(sum + 4.5);`

In Java, **an augmented expression** of the form:

$x1 \text{ op} x2$

is implemented as:

$x1 = (T)(x1 \text{ op} x2)$

Where: **T** is the type for **$x1$** .

range increases



byte, short, int, long, float, double

Problem: Displaying Time

Write a program that obtains minutes and remaining seconds from seconds.

DisplayTime

Problem: Keeping Two Digits After Decimal Points

Write a program that displays the sales tax with two digits after the decimal point.

SalesTax

The boolean Type and Relational Operators

Java provides six comparison operators (*relational operators*) that can be used to compare two values. The result of the comparison is a Boolean value: `true` or `false`.

EX: `boolean b = (1 > 2) ;`

Java Operator	Mathematics Symbol	Name	Example (radius is 5)	Result
<code><</code>	<code><</code>	less than	<code>radius < 0</code>	<code>false</code>
<code><=</code>	<code>≤</code>	less than or equal to	<code>radius <= 0</code>	<code>false</code>
<code>></code>	<code>></code>	greater than	<code>radius > 0</code>	<code>true</code>
<code>>=</code>	<code>≥</code>	greater than or equal to	<code>radius >= 0</code>	<code>true</code>
<code>==</code>	<code>=</code>	equal to	<code>radius == 0</code>	<code>false</code>
<code>!=</code>	<code>≠</code>	not equal to	<code>radius != 0</code>	<code>true</code>

Logical Operators

Operator	Name	Description
!	not	logical negation
&&	and	logical conjunction
	or	logical disjunction
^	exclusive or	logical exclusion

Examples

Here is a program that checks whether a number is divisible by 2 and 3, whether a number is divisible by 2 or 3, and whether a number is divisible by 2 or 3 but not both:

TestBooleanOperators

Operator Precedence and Associativity

The expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

If operators with the same precedence are next to each other, their associativity determines the order of evaluation. All binary operators except assignment operators are left-associative.

Operator Associativity

When two operators with the same precedence are evaluated, the *associativity* of the operators determines the order of evaluation. All binary operators except assignment operators are *left-associative*.

$a - b + c - d$ is equivalent to $((a - b) + c) - d$

Assignment operators are *right-associative*. Therefore, the expression

$a = b += c = 5$ is equivalent to $a = (b += (c = 5))$

Character Data Type

- ❑ Java supports *Unicode* to support the written texts in the world's diverse languages.
- ❑ Unicode was originally designed as a **16-bit character** encoding. The primitive data type **char** was intended to take advantage of this design by providing a simple data type that could hold any character.
- ❑ However, it turned out that the 65,536 characters possible in a 16-bit encoding are not sufficient to represent all the characters in the world.
- ❑ The Unicode standard therefore has been extended to allow up to 1,112,064 characters. Those characters that go beyond the original 16-bit limit are called *supplementary characters*. Java supports the supplementary characters.

Character Data Type

Unicode takes two bytes, preceded by `\u`, expressed in four hexadecimal numbers:

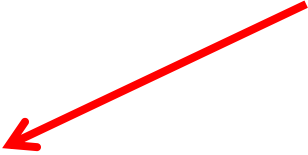
```
char letter = 'A';
```

```
char numChar = '4';
```

```
char letter = '\u0041'; (Unicode)
```

```
char numChar = '\u0034'; (Unicode)
```

Four hexadecimal digits.



NOTE: The increment and decrement operators can also be used on char variables to get the next or preceding Unicode character.

EX: `char ch = 'a';`

```
System.out.println(++ch);
```

Code for Commonly Used Characters

Characters	Code Value in Decimal	Unicode Value
'0' to '9'	48 to 57	\u0030 to \u0039
'A' to 'Z'	65 to 90	\u0041 to \u005A
'a' to 'z'	97 to 122	\u0061 to \u007A

Escape Sequences for Special Characters

<i>Escape Sequence</i>	<i>Name</i>	<i>Unicode Code</i>	<i>Decimal Value</i>
<code>\b</code>	Backspace	<code>\u0008</code>	8
<code>\t</code>	Tab	<code>\u0009</code>	9
<code>\n</code>	Linefeed	<code>\u000A</code>	10
<code>\f</code>	Formfeed	<code>\u000C</code>	12
<code>\r</code>	Carriage Return	<code>\u000D</code>	13
<code>\\</code>	Backslash	<code>\u005C</code>	92
<code>\"</code>	Double Quote	<code>\u0022</code>	34

Casting between char and Numeric Types

```
int i = 'a'; // Same as int i = (int) 'a';
```

```
char c = 97; // Same as char c = (char) 97;
```

Comparing and Testing Characters

```
if (ch >= 'A' && ch <= 'Z')
```

```
    System.out.println(ch + " is an uppercase letter");
```

```
else if (ch >= 'a' && ch <= 'z')
```

```
    System.out.println(ch + " is a lowercase letter");
```

```
else if (ch >= '0' && ch <= '9')
```

```
    System.out.println(ch + " is a numeric character");
```

Methods in the Character Class

Method	Description
<code>isDigit(ch)</code>	Returns true if the specified character is a digit.
<code>isLetter(ch)</code>	Returns true if the specified character is a letter.
<code>isLetterOfDigit(ch)</code>	Returns true if the specified character is a letter or digit.
<code>isLowerCase(ch)</code>	Returns true if the specified character is a lowercase letter.
<code>isUpperCase(ch)</code>	Returns true if the specified character is an uppercase letter.
<code>toLowerCase(ch)</code>	Returns the lowercase of the specified character.
<code>toUpperCase(ch)</code>	Returns the uppercase of the specified character.

The String Type

The char type only represents one character. To represent a string of characters, use the data type called String. For example,

```
String message = "Welcome to Java";
```

String is a predefined class in the Java library just like the System class and Scanner class. The String type is not a primitive type. It is known as a *reference type*. Any Java class can be used as a reference type for a variable.

Strings are objects in Java. String methods can only be invoked from a specific string instance.

```
System.out.println("The length of " + message  
+ " is " + message.length());
```

String Concatenation

```
String s3 = s1.concat(s2);    or    String s3 = s1 + s2;
```

// Three strings are concatenated

```
String message = "Welcome " + "to " + "Java";
```

// String Chapter is concatenated with number 2

```
String s = "Chapter" + 2;
```

// String Supplement is concatenated with character B

```
String s1 = "Supplement" + 'B';
```

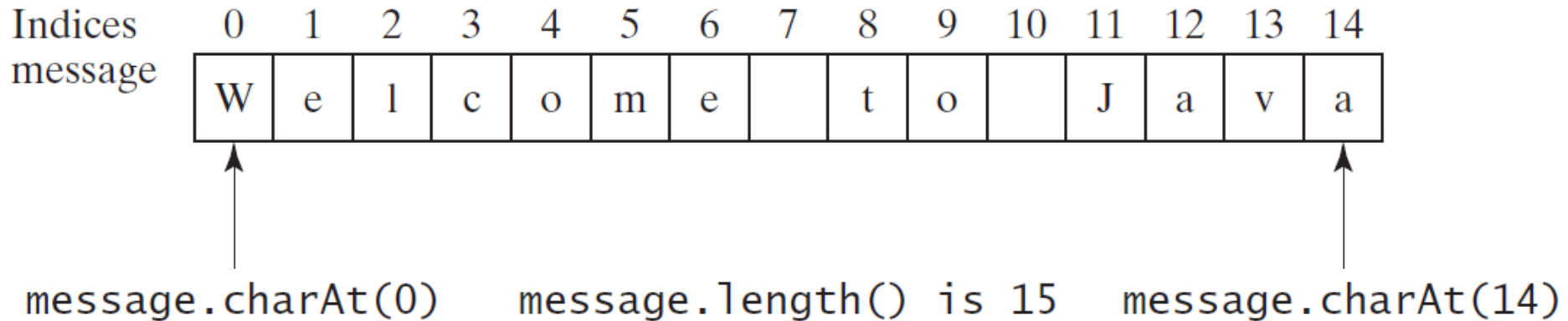
//Convert number to String

```
String s = number + "";
```

Simple Methods for **String** Objects

Method	Description
<code>length()</code>	Returns the number of characters in this string.
<code>charAt(index)</code>	Returns the character at the specified index from this string.
<code>concat(s1)</code>	Returns a new string that concatenates this string with string s1.
<code>toUpperCase()</code>	Returns a new string with all letters in uppercase.
<code>toLowerCase()</code>	Returns a new string with all letters in lowercase.
<code>trim()</code>	Returns a new string with whitespace characters trimmed on both sides.

Getting Characters from a String



```
String message = "Welcome to Java";  
System.out.println("The first character in message is "  
    + message.charAt(0) );
```

Reading a Character from the Console

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter a character: ");  
String s = input.nextLine();  
char ch = s.charAt(0);  
System.out.println("The character entered is " + ch);
```

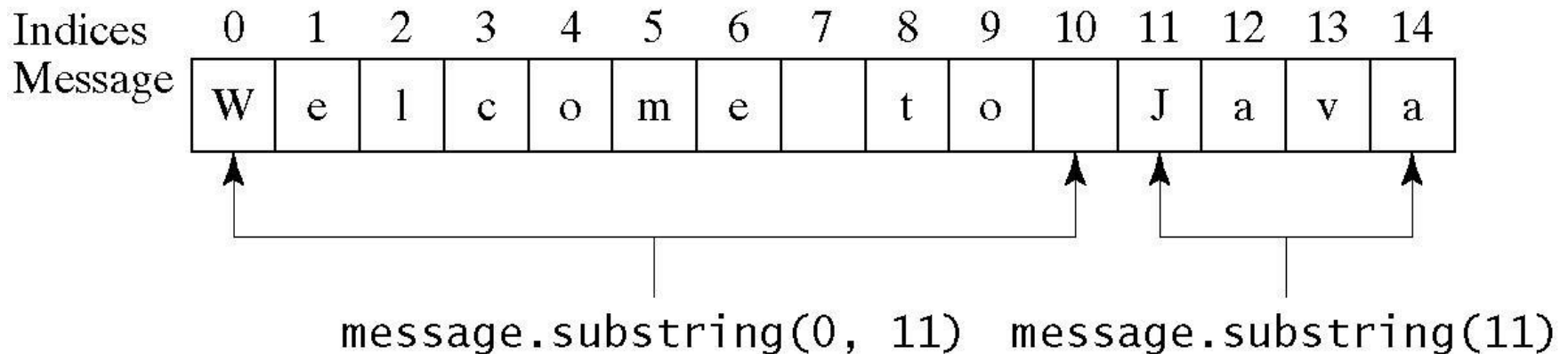
Comparing Strings

Method	Description
<code>equals(s1)</code>	Returns true if this string is equal to string <code>s1</code> .
<code>equalsIgnoreCase(s1)</code>	Returns true if this string is equal to string <code>s1</code> ; it is case insensitive.
<code>compareTo(s1)</code>	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than <code>s1</code> .
<code>compareToIgnoreCase(s1)</code>	Same as <code>compareTo</code> except that the comparison is case insensitive.
<code>startsWith(prefix)</code>	Returns true if this string starts with the specified prefix.
<code>endsWith(suffix)</code>	Returns true if this string ends with the specified suffix.

OrderTwoCities

Obtaining Substrings

Method	Description
<code>substring(beginIndex)</code>	Returns this string's substring that begins with the character at the specified <code>beginIndex</code> and extends to the end of the string, as shown in Figure 4.2.
<code>substring(beginIndex, endIndex)</code>	Returns this string's substring that begins at the specified <code>beginIndex</code> and extends to the character at index <code>endIndex - 1</code> , as shown in Figure 9.6. Note that the character at <code>endIndex</code> is not part of the substring.

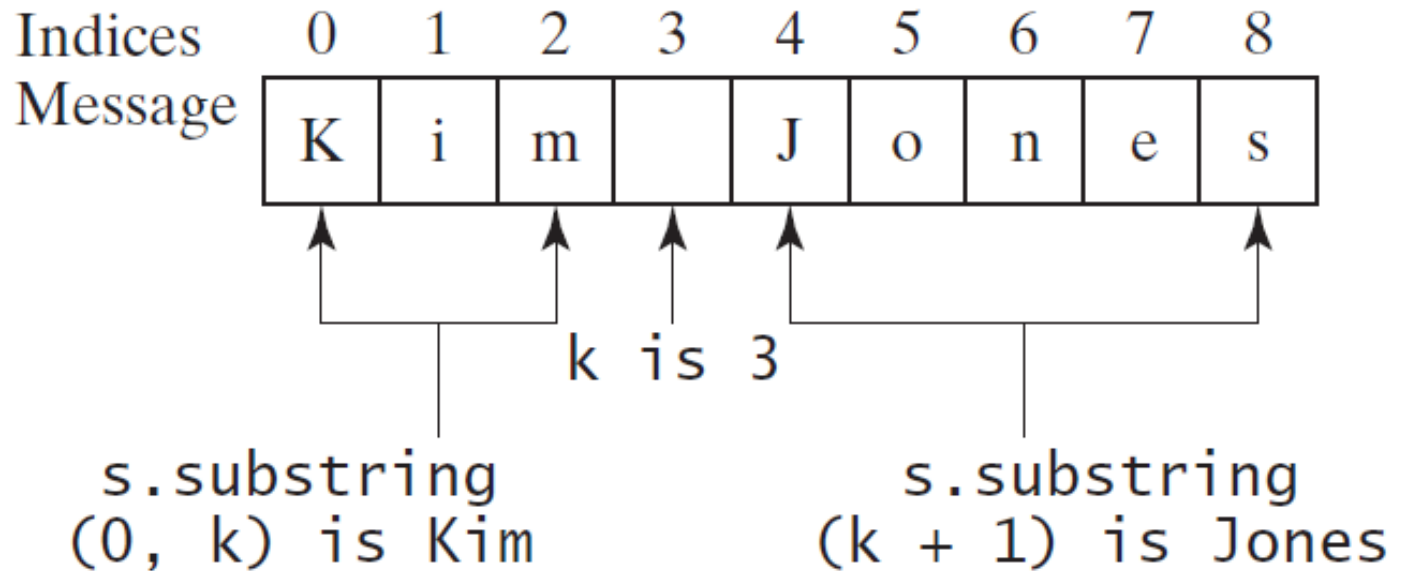


Finding a Character or a Substring in a String

Method	Description
<code>indexOf(ch)</code>	Returns the index of the first occurrence of <code>ch</code> in the string. Returns <code>-1</code> if not matched.
<code>indexOf(ch, fromIndex)</code>	Returns the index of the first occurrence of <code>ch</code> after <code>fromIndex</code> in the string. Returns <code>-1</code> if not matched.
<code>indexOf(s)</code>	Returns the index of the first occurrence of string <code>s</code> in this string. Returns <code>-1</code> if not matched.
<code>indexOf(s, fromIndex)</code>	Returns the index of the first occurrence of string <code>s</code> in this string after <code>fromIndex</code> . Returns <code>-1</code> if not matched.
<code>lastIndexOf(ch)</code>	Returns the index of the last occurrence of <code>ch</code> in the string. Returns <code>-1</code> if not matched.
<code>lastIndexOf(ch, fromIndex)</code>	Returns the index of the last occurrence of <code>ch</code> before <code>fromIndex</code> in this string. Returns <code>-1</code> if not matched.
<code>lastIndexOf(s)</code>	Returns the index of the last occurrence of string <code>s</code> . Returns <code>-1</code> if not matched.
<code>lastIndexOf(s, fromIndex)</code>	Returns the index of the last occurrence of string <code>s</code> before <code>fromIndex</code> . Returns <code>-1</code> if not matched.

Finding a Character or a Substring in a String

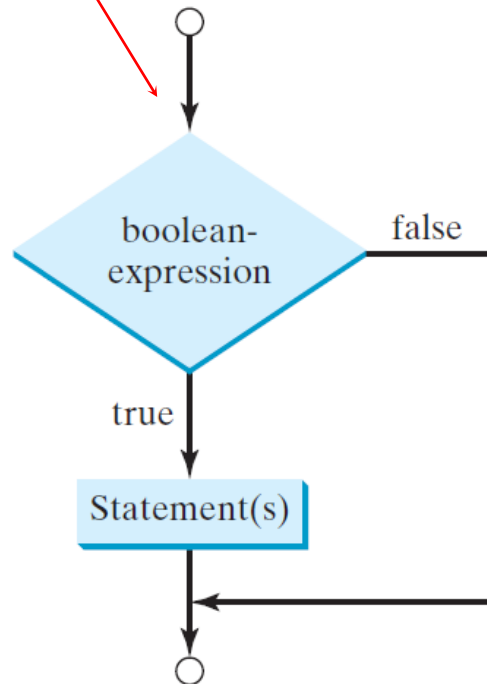
```
int k = s.indexOf(' ');  
String firstName = s.substring(0, k);  
String lastName = s.substring(k + 1);
```



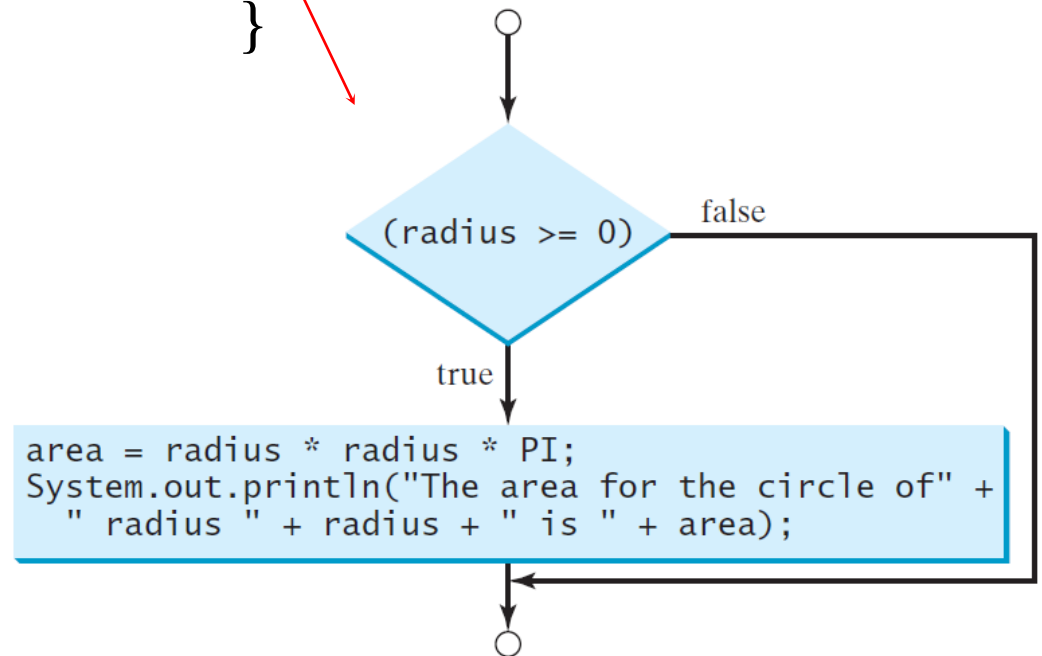
JAVA Selections

One-way if Statements

```
if (boolean-expression) {  
    statement(s);  
}
```

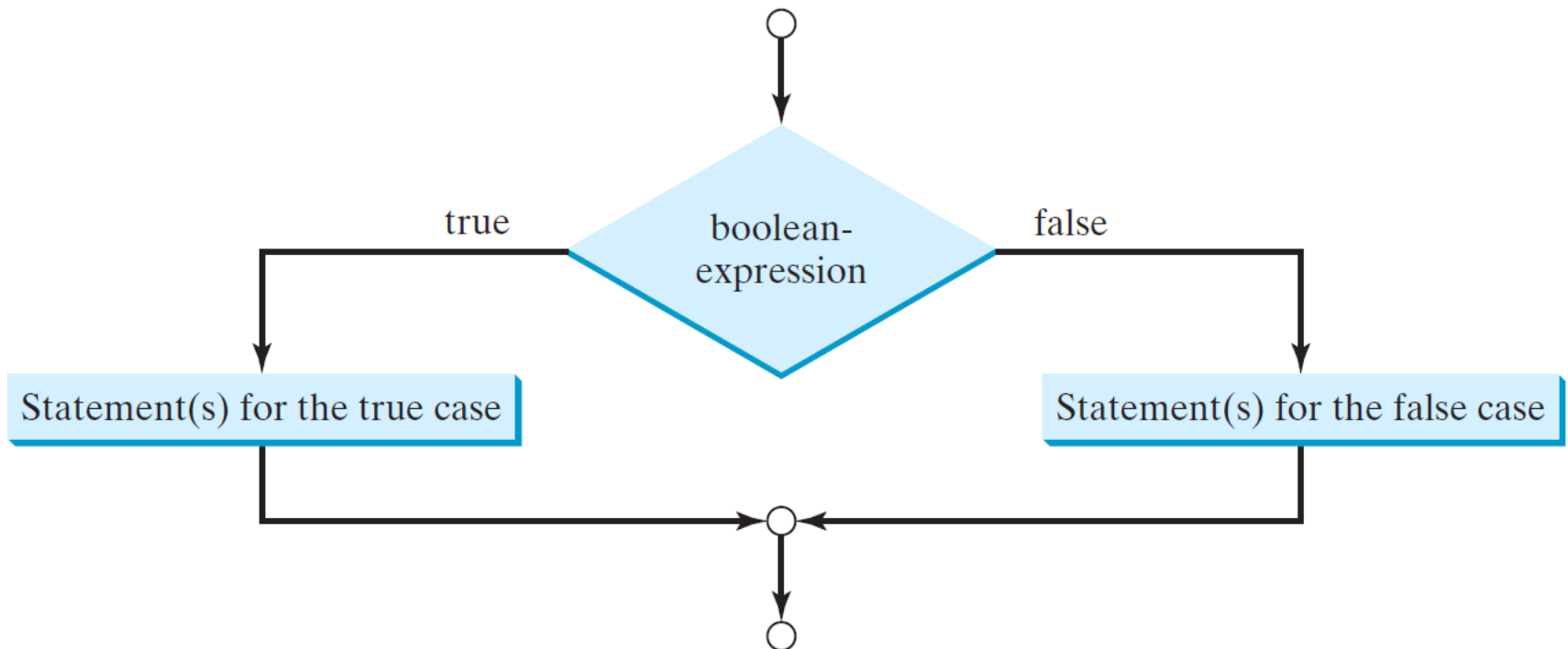


```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area"  
        + " for the circle of radius "  
        + radius + " is " + area);  
}
```



The Two-way `if` Statement

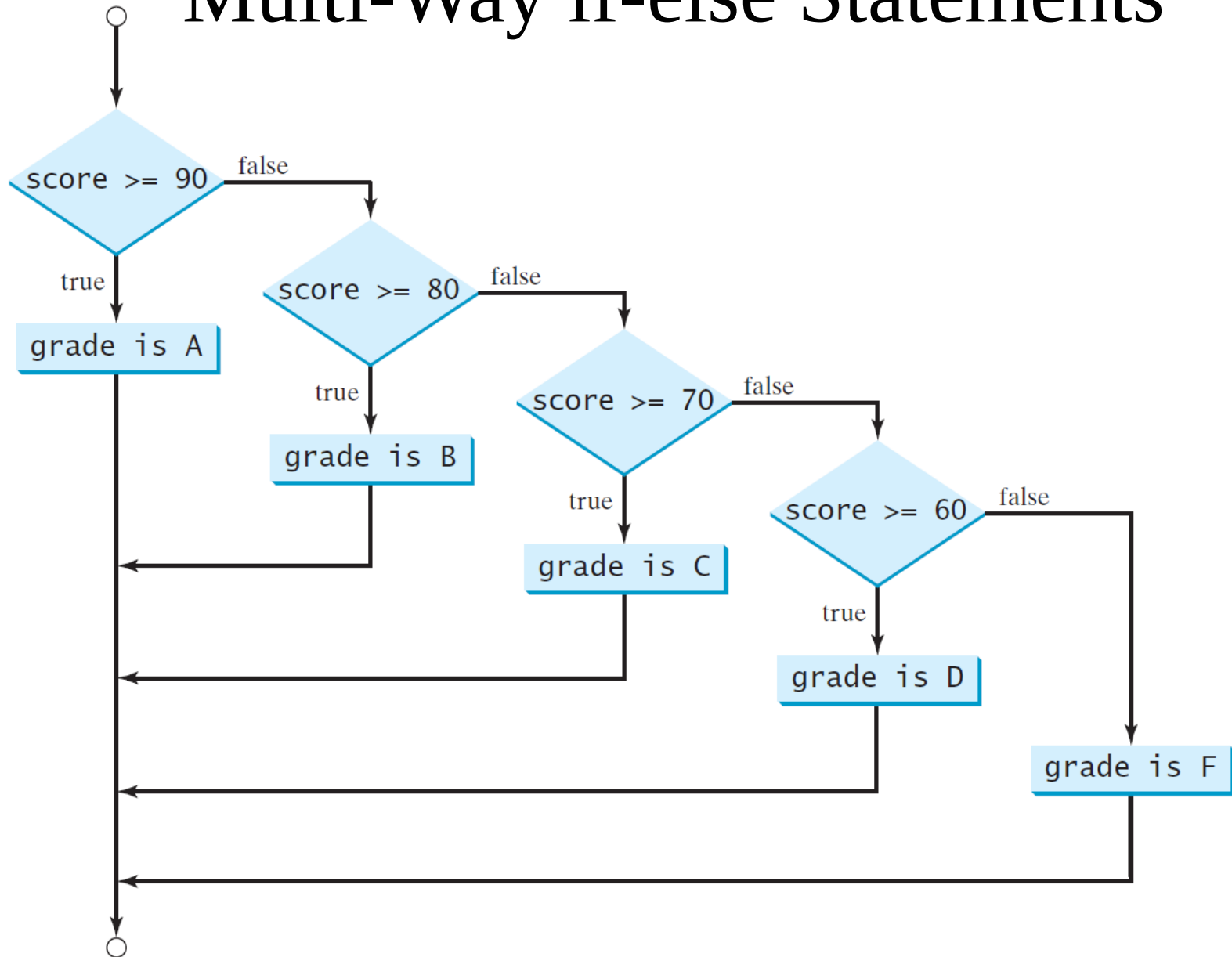
```
if (boolean-expression) {  
    statement(s)-for-the-true-case;  
}  
else {  
    statement(s)-for-the-false-case;  
}
```



if-else Example

```
if (radius >= 0) {  
    area = radius * radius * 3.14159;  
  
    System.out.println("The area for the "  
        + "circle of radius " + radius +  
        " is " + area);  
}  
else {  
    System.out.println("Negative input");  
}
```

Multi-Way if-else Statements



Multi-Way if-else Statements

```
if (score >= 90.0)
    System.out.print("A");
else
    if (score >= 80.0)
        System.out.print("B");
    else
        if (score >= 70.0)
            System.out.print("C");
        else
            if (score >= 60.0)
                System.out.print("D");
            else
                System.out.print("F");
```

(a)

Equivalent

This is better

```
if (score >= 90.0)
    System.out.print("A");
else if (score >= 80.0)
    System.out.print("B");
else if (score >= 70.0)
    System.out.print("C");
else if (score >= 60.0)
    System.out.print("D");
else
    System.out.print("F");
```

(b)

Note

The else clause matches the most recent if clause in the same block. To force the else clause to match an outer if clause, you must add a pair of braces:

EX:

```
int i = 1;    int j = 2;    int k = 3;
if (i > j) {
    if (i > k)
        System.out.println("A");
}
else
    System.out.println("B");
```


Common Errors

Adding a semicolon at the end of an if clause is a common mistake.

```
if (radius >= 0);  Wrong
{
    area = radius*radius*PI;
    System.out.println(
        "The area for the circle of radius " +
        radius + " is " + area);
}
```

This mistake is hard to find, because it is not a compilation error or a runtime error, it is a logic error.

This error often occurs when you use the next-line block style.

TIP

```
if (number % 2 == 0)
    even = true;
else
    even = false;
```

Equivalent

```
boolean even
    = number % 2 == 0;
```

```
if (even == true)
    System.out.println(
        "It is even.");
```

Equivalent

```
if (even)
    System.out.println(
        "It is even.");
```

Problem: An Improved Math Learning Tool

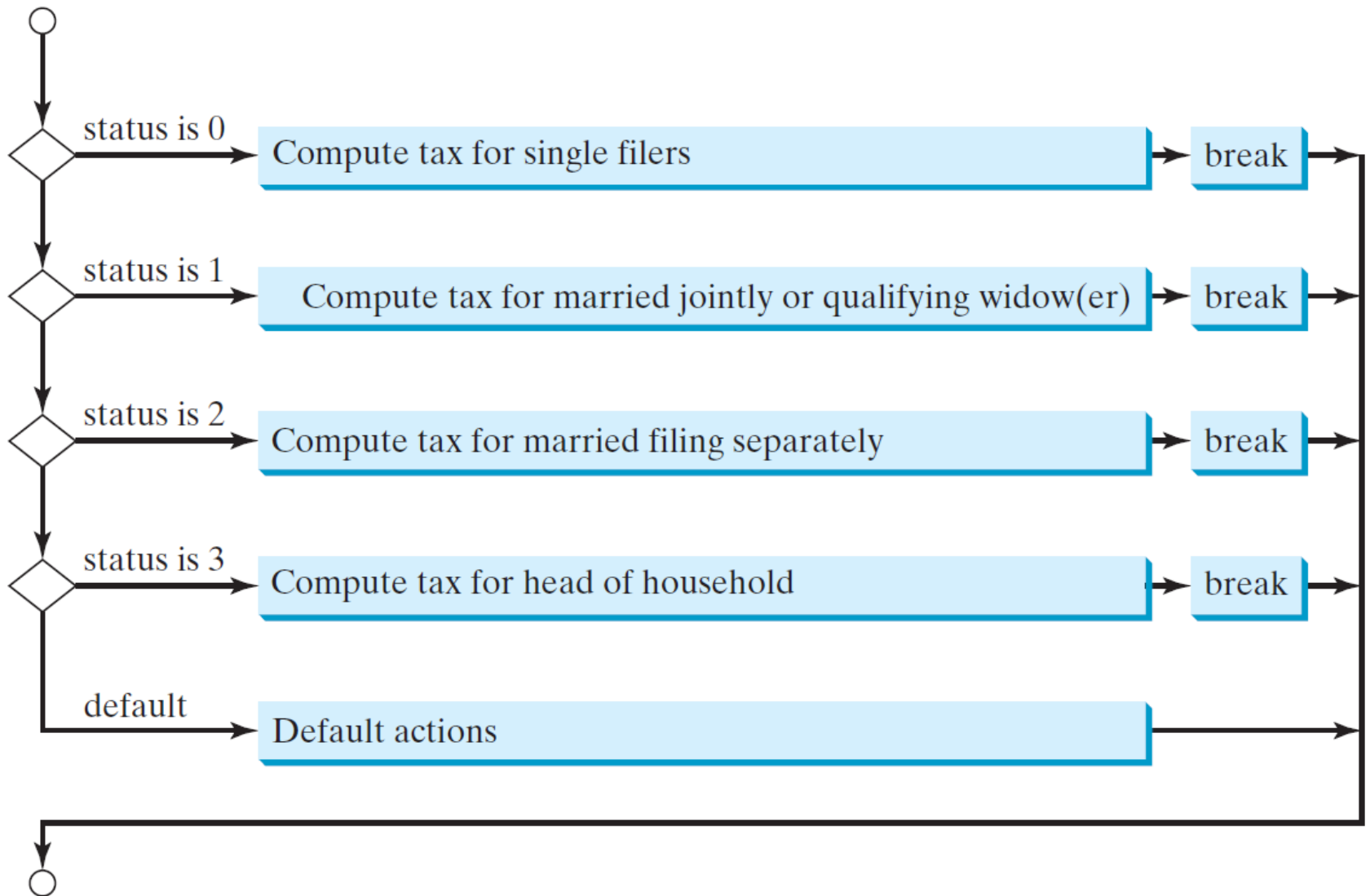
This example creates a program to teach a first grade child how to learn subtractions. The program randomly generates two single-digit integers number1 and number2 with number1 $\geq$ number2 and displays a question such as “What is $9 - 2$?” to the student. After the student types the answer, the program displays whether the answer is correct.

SubtractionQuiz

switch Statements

```
switch (status) {  
    case 0: compute taxes for single filers;  
            break;  
    case 1: compute taxes for married file jointly;  
            break;  
    case 2: compute taxes for married file separately;  
            break;  
    case 3: compute taxes for head of household;  
            break;  
    default: System.out.println("Errors: invalid status");  
            System.exit(1);  
}
```

switch Statement Flow Chart

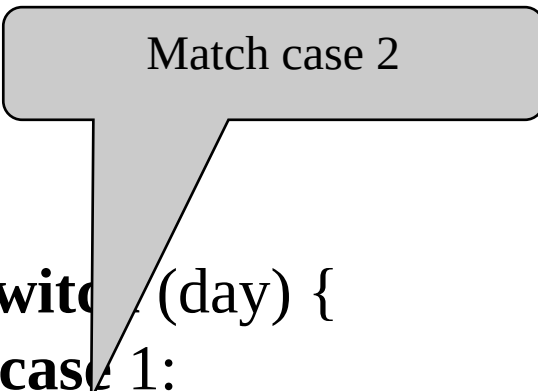


Trace switch statement

Suppose day is 2:

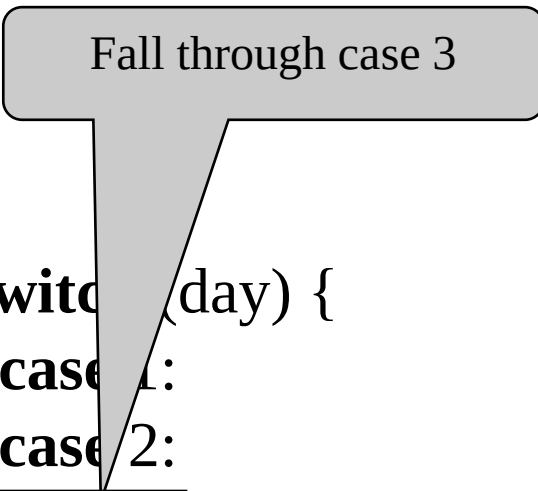
```
switch (day) {  
  case 1:  
  case 2:  
  case 3:  
  case 4:  
  case 5: System.out.println("Weekday"); break;  
  case 0:  
  case 6: System.out.println("Weekend");  
}
```

Trace switch statement



```
switch (day) {  
  case 1:  
  case 2:  
  case 3:  
  case 4:  
  case 5: System.out.println("Weekday"); break;  
  case 0:  
  case 6: System.out.println("Weekend");  
}
```

Trace switch statement

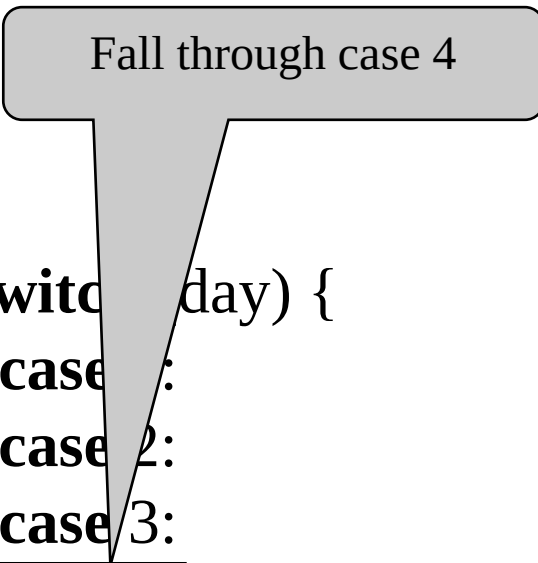


Fall through case 3

```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

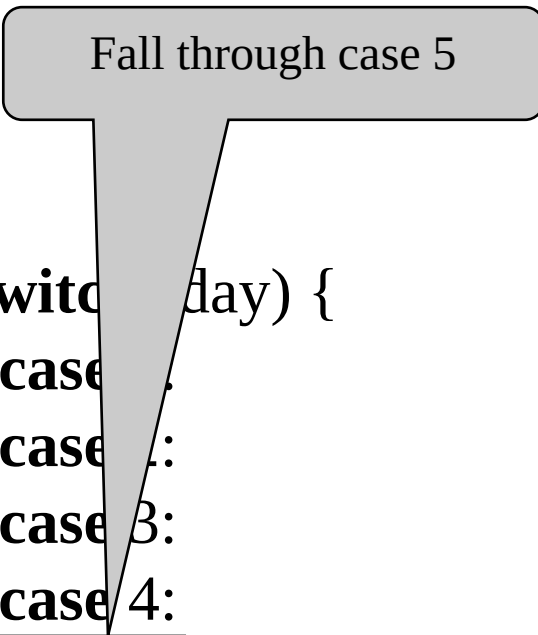
The diagram illustrates a fall-through case in a switch statement. A callout box labeled "Fall through case 3" points to the "case 3:" line in the code. The code shows a switch statement for "day" with cases 1 through 6. Case 3 is highlighted with a box, and the code continues to case 4, case 5 (which has a break statement), case 0, and case 6.

Trace switch statement



```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

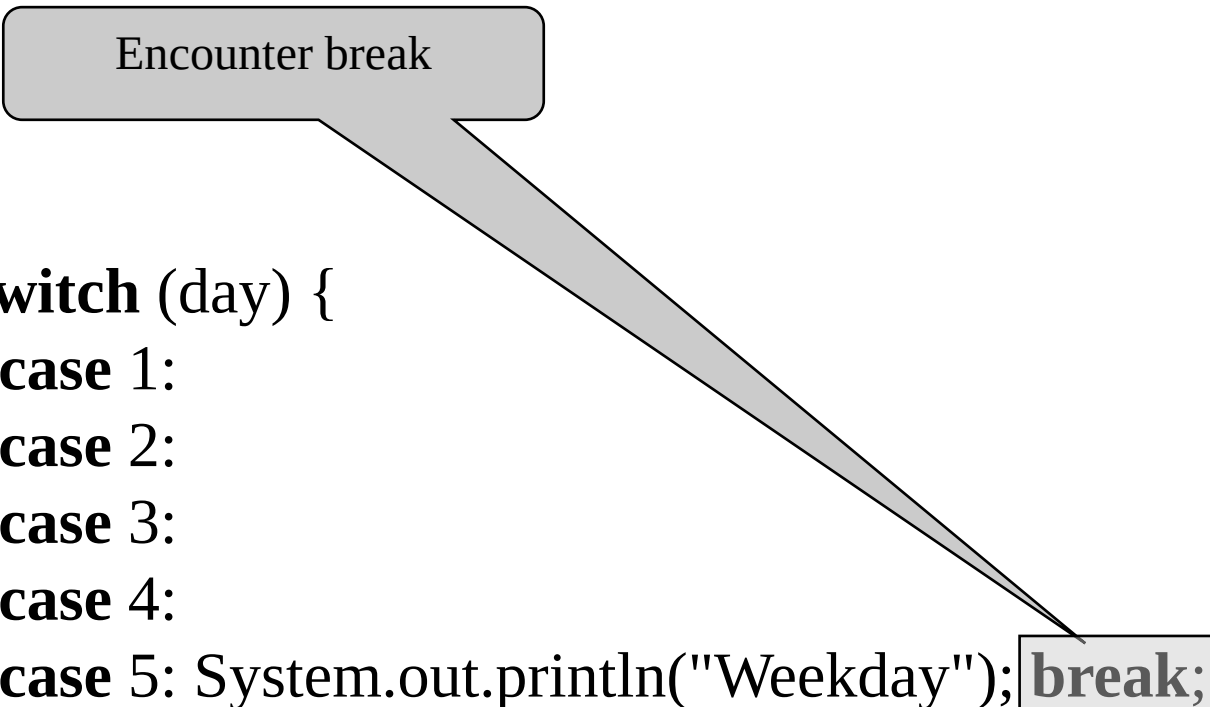
Trace switch statement



```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Trace switch statement

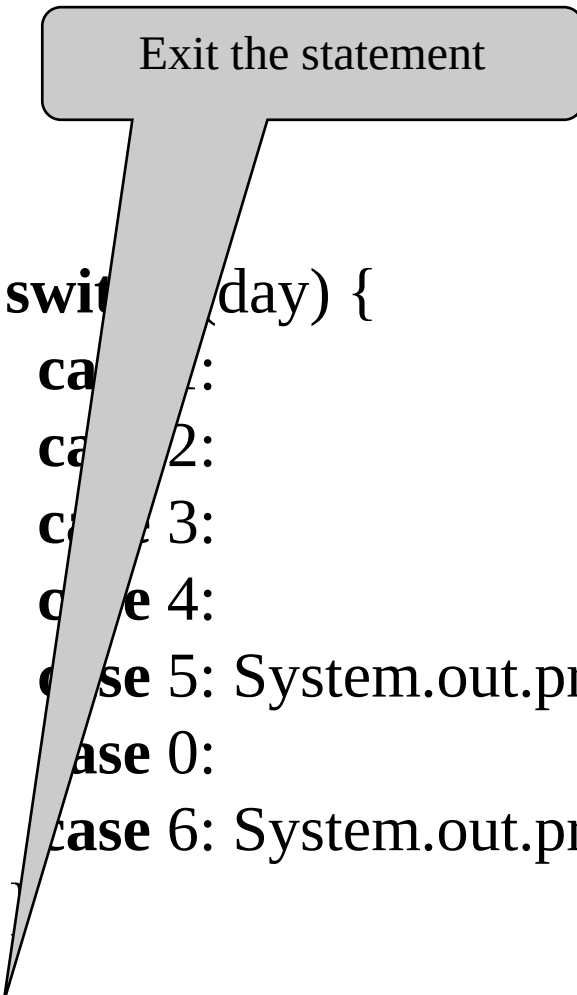
Encounter break



```
switch (day) {  
  case 1:  
  case 2:  
  case 3:  
  case 4:  
  case 5: System.out.println("Weekday"); break;  
  case 0:  
  case 6: System.out.println("Weekend");  
}
```

Trace switch statement

Exit the statement



```
switch (day) {  
    case 1:  
    case 2:  
    case 3:  
    case 4:  
    case 5: System.out.println("Weekday"); break;  
    case 0:  
    case 6: System.out.println("Weekend");  
}
```

Conditional Operators

(boolean-expression) ? expression1 : expression2

Also known as a ternary operator

Ex 1:

```
y = (x > 0) ? 1 : -1;
```

is equivalent to:

```
if (x > 0)    y = 1 else    y = -1;
```

Ex 2:

```
System.out.println((num % 2 == 0) ?  
    num + "is even" : num + "is odd");
```

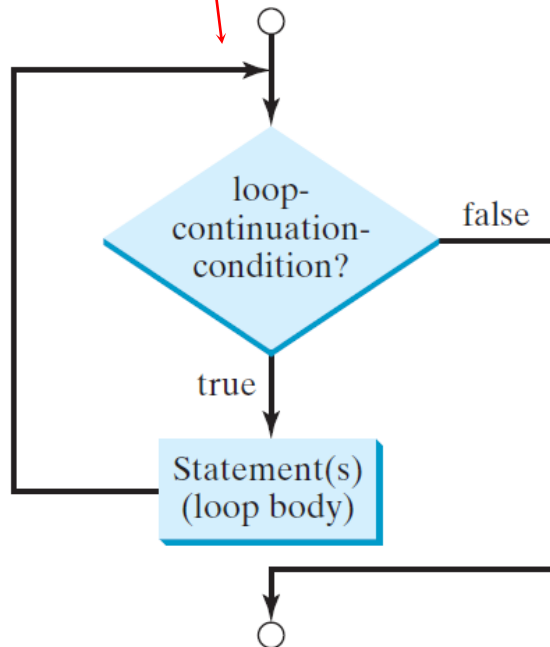
is equivalent to:

```
if (num % 2 == 0)  
    System.out.println(num + "is even");  
else  
    System.out.println(num + "is odd");
```

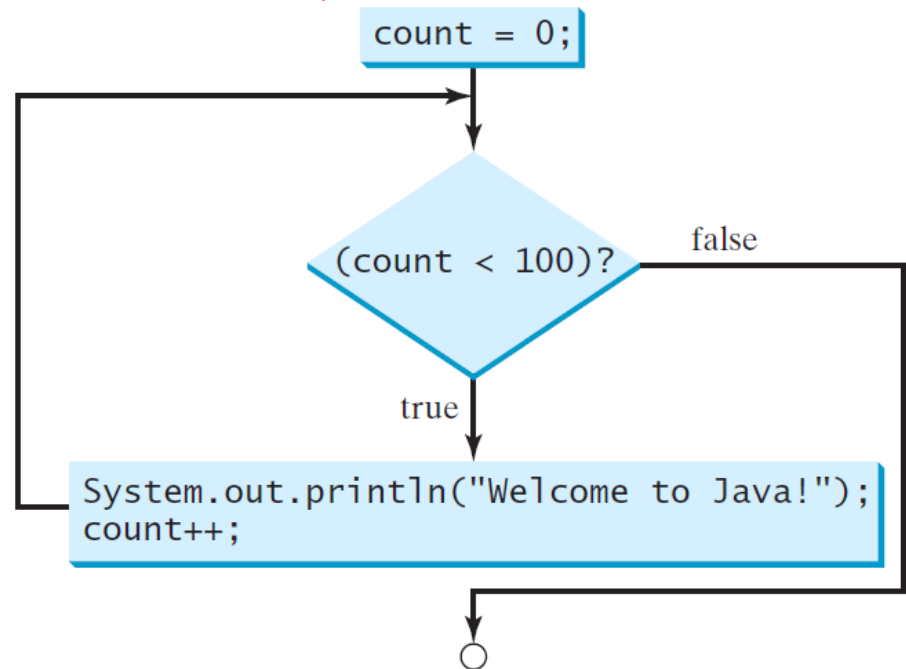
JAVA Loops

while Loop Flow Chart

```
while (loop-continuation-condition) {  
    // loop-body;  
    Statement(s);  
}
```



```
int count = 0;  
while (count < 100) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```



Trace while Loop

```
int count = 0;
```

Initialize count

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```


Trace while Loop, cont.

```
int count = 0;
```

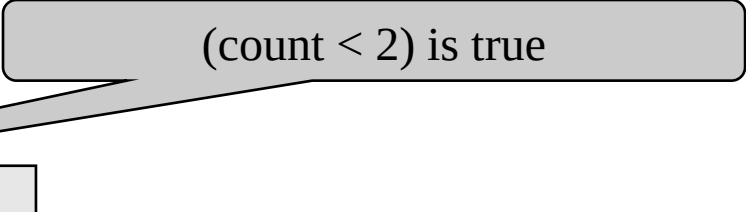
```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```

(count < 2) is true

A gray callout box with a black border and rounded corners contains the text "(count < 2) is true". A black arrow points from the left side of this box to the condition "(count < 2)" in the while loop header of the code block below.

Trace while Loop, cont.

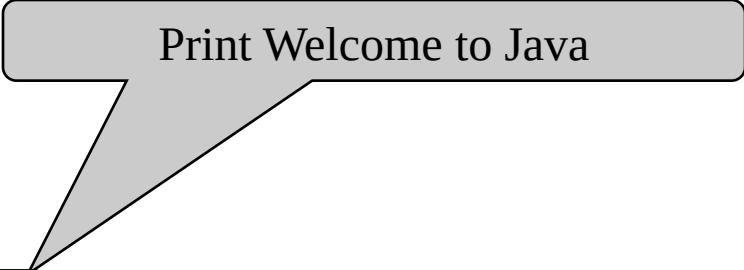
```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

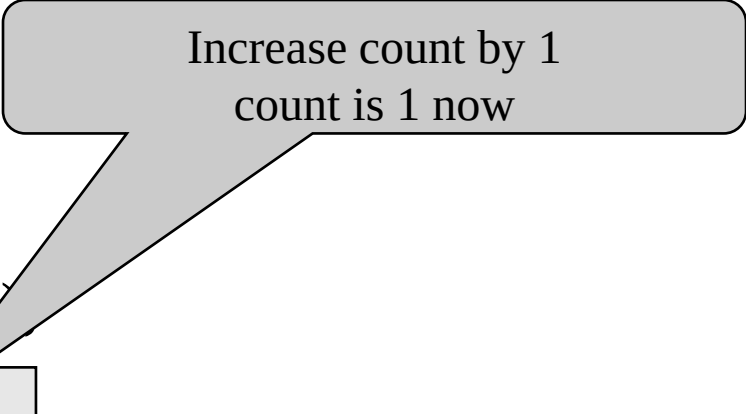
```
}
```



Print Welcome to Java

Trace while Loop, cont.

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!")  
    count++;  
}
```



Increase count by 1
count is 1 now

Trace while Loop, cont.

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```

(count < 2) is still true since count
is 1

Trace while Loop, cont.

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```



Print Welcome to Java

Trace while Loop, cont.

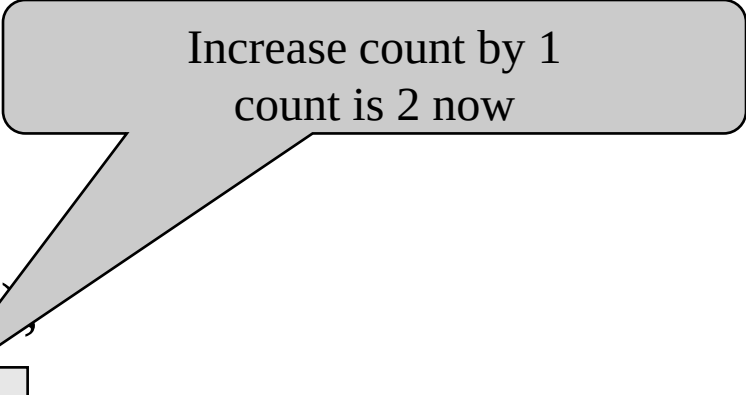
```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

```
}
```



Increase count by 1
count is 2 now

Trace while Loop, cont.

```
int count = 0;
```

```
while (count < 2) {
```

```
    System.out.println("Welcome to Java!");
```

```
    count++;
```

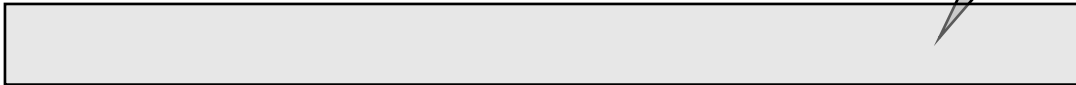
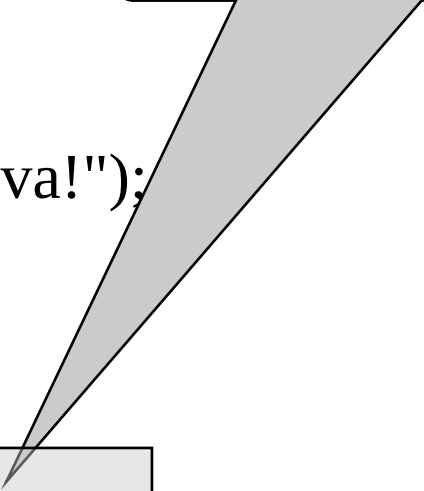
```
}
```

(count < 2) is false since count is 2
now

Trace while Loop

```
int count = 0;  
while (count < 2) {  
    System.out.println("Welcome to Java!");  
    count++;  
}
```

The loop exits. Execute the next statement after the loop.



Ending a Loop with a Sentinel Value

Often the number of times a loop is executed is not predetermined. You may use an input value to signify the end of the loop. Such a value is known as a *sentinel value*.

Write a program that reads and calculates the sum of an unspecified number of integers. The input 0 signifies the end of the input.

SentinelValue

Caution

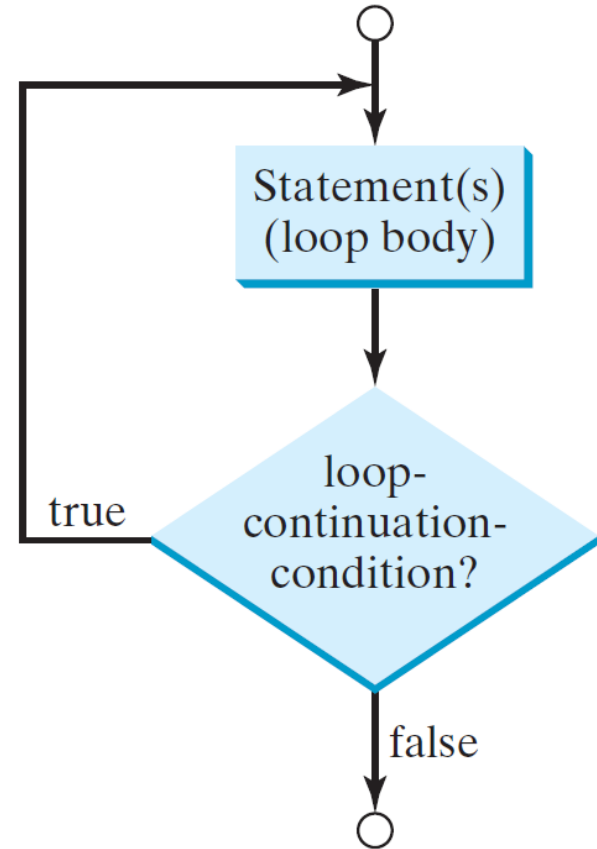
Don't use floating-point values for equality checking in a loop control. Since floating-point values are approximations for some values, using them could result in imprecise counter values and inaccurate results.

EX: Consider the following code for computing

$$1 + 0.9 + 0.8 + \dots + 0.1$$

```
double item = 1;           double sum = 0;
while (item != 0) //No guarantee item will be 0
{
    sum += item;
    item -= 0.1;
}
System.out.println(sum);
```

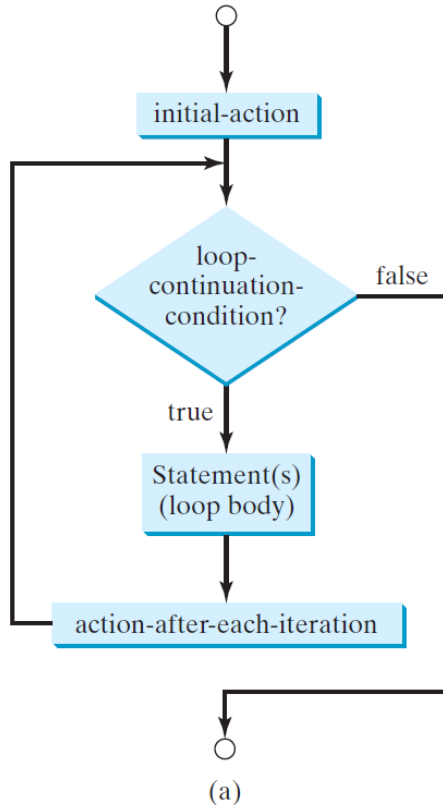
do-while Loop



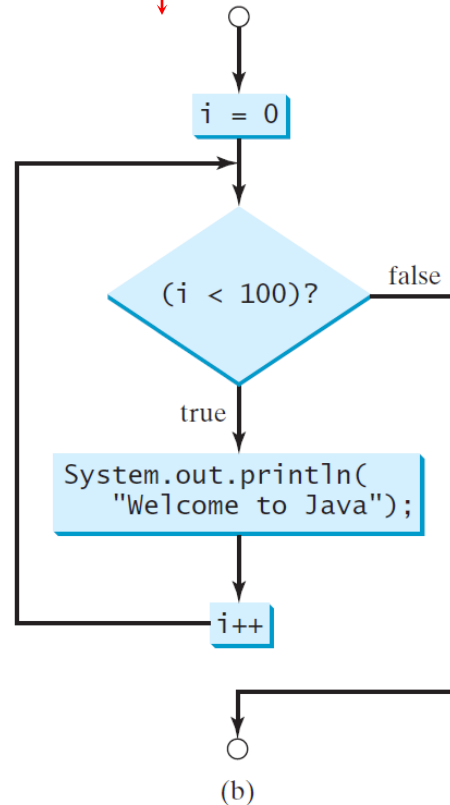
```
do {  
    // Loop body;  
    Statement(s) ;  
} while (loop-continuation-condition) ;
```

for Loops

```
for (initial-action; loop-  
    continuation-condition; action-  
    after-each-iteration) {  
    // loop body;  
    Statement(s);  
}
```



```
int i;  
for (i = 0; i < 100; i++) {  
    System.out.println(  
        "Welcome to Java!");  
}
```



Trace for Loop

```
int i;
```

Declare i

```
for (i = 0; i < 2; i++) {  
    System.out.println(  
        "Welcome to Java!");  
}
```

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute initializer
i is now 0

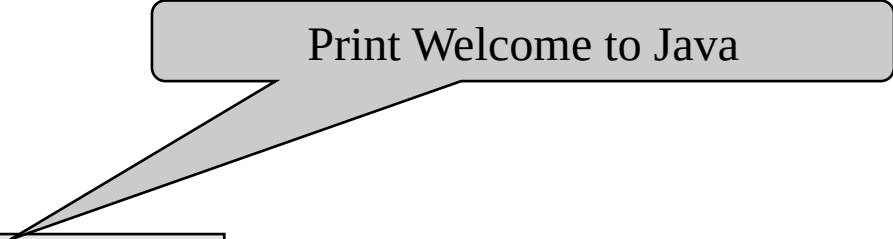
Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println( "Welcome to Java!");  
}
```

(i < 2) is true
since i is 0

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```



Print Welcome to Java

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute adjustment statement
i now is 1

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is still true
since i is 1

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```



Print Welcome to Java

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

Execute adjustment statement
i now is 2

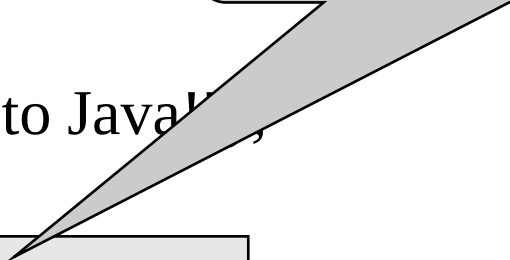
Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java!");  
}
```

(i < 2) is false
since i is 2

Trace for Loop, cont.

```
int i;  
for (i = 0; i < 2; i++) {  
    System.out.println("Welcome to Java");  
}
```



Exit the loop. Execute the next statement after the loop

Note

The initial-action in a for loop can be a list of zero or more comma-separated expressions. The action-after-each-iteration in a for loop can be a list of zero or more comma-separated statements. Therefore, the following two for loops are correct. They are rarely used in practice, however.

```
for (int i = 1; i < 100; System.out.println(i++));
```

```
for (int i = 0, j = 0; (i + j < 10); i++, j++) {
```

```
    // Do something
```

```
}
```

Note

If the loop-continuation-condition in a for loop is omitted, it is implicitly true. Thus the statement given below in (a), which is an infinite loop, is correct. Nevertheless, it is better to use the equivalent loop in (b) to avoid confusion:

```
for ( ; ; ) {  
    // Do something  
}
```

(a)

Equivalent

```
while (true) {  
    // Do something  
}
```

(b)

Caution

Adding a semicolon at the end of the for clause before the loop body is a common mistake, as shown below:

```
for (int i=0; i<10; i++) ;  
{  
    System.out.println("i is " + i);  
}
```

Logic
Error



Caution, cont.

Similarly, the following loop is also wrong:

```
int i=0;
while (i < 10);  Logic Error
{
    System.out.println("i is " + i);
    i++;
}
```

In the case of the do loop, the following semicolon is needed to end the loop.

```
int i=0;
do {
    System.out.println("i is " + i);
    i++;
} while (i<10);  Correct
```

Using `break` and `continue`

Examples for using the `break` and `continue` keywords:

- `TestBreak.java`

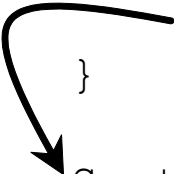
`TestBreak`

- `TestContinue.java`

`TestContinue`

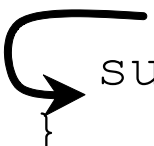
break

```
public class TestBreak {  
    public static void main(String[] args) {  
        int sum = 0;  
        int number = 0;  
  
        while (number < 20) {  
            number++;  
            sum += number;  
            if (sum >= 100)  
                break;  
        }  
        System.out.println("The number is " + number);  
        System.out.println("The sum is " + sum);  
    }  
}
```



continue

```
public class TestContinue {  
    public static void main(String[] args) {  
        int sum = 0;  
        int number = 0;  
  
        while (number < 20) {  
            number++;  
            if (number == 10 || number == 11)  
                continue;  
            sum += number;  
        }  
  
        System.out.println("The sum is " + sum);  
    }  
}
```



Thank you
Thank you

